

Probeklausur V2 — 06.03.2026

Aufgabe 1 (47.25 Punkte)

Implementiere die Klassen

- **Boardable** (1.5 Punkte),
- **SeatType** (3.5 Punkte),
- **Passenger** (3.5 Punkte),
- **Row** (10.0 Punkte),
- **FirstClassRow** (5.75 Punkte),
- **EmergencyRow** (6.0 Punkte),
- **Airplane** (12.0 Punkte) und
- **ExamTask** (5.0 Punkte)

entsprechend dem Klassendiagramm.

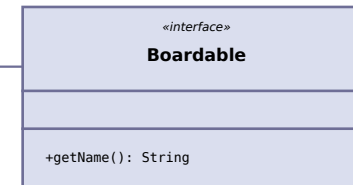
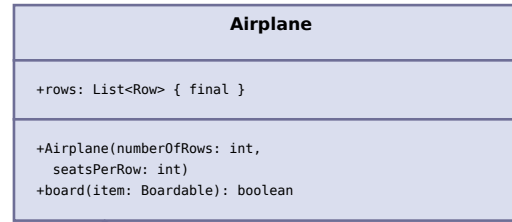
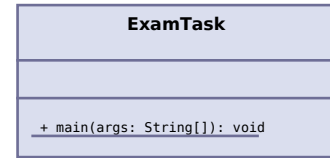
Befolge alle Hinweise bei der Implementierung!

Die Klasse FlightCrew ist gegeben und muss nicht implementiert werden.

Glossar

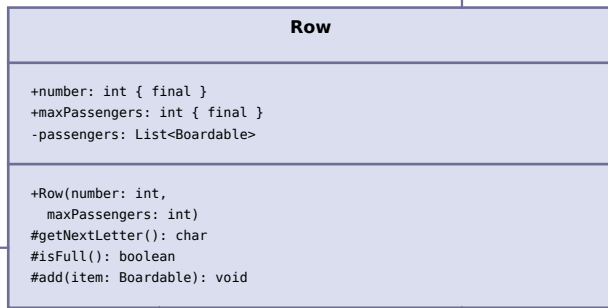
Englisch	Deutsch
airplane	Flugzeug
aisle	Gang
board	einsteigen
boardable	einsteigefaeig
emergency row	Notausgangreihe
first class	First Class
flight crew	Flugbesatzung
passenger	Passagier
preferred	bevorzugt
row	Reihe
seat type	Sitztyp
window	Fenster

Klassendiagramm



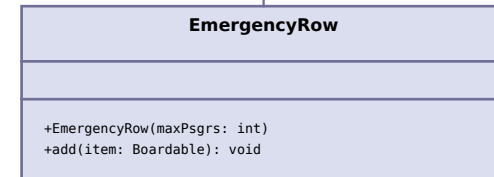
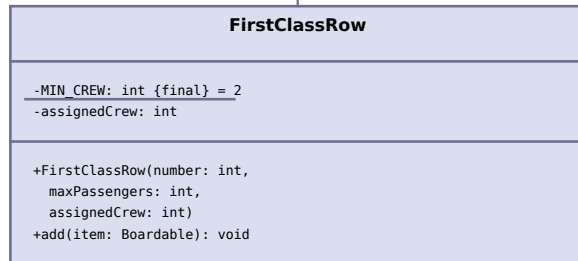
implements

implements



extends

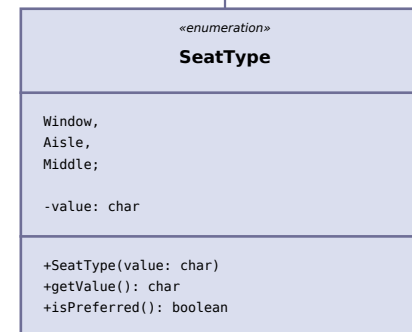
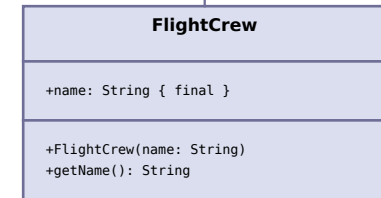
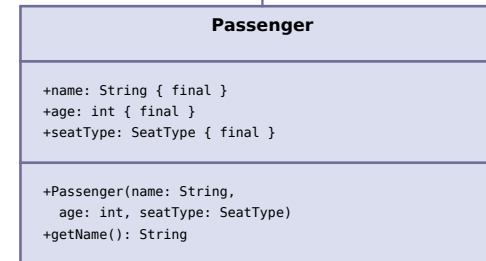
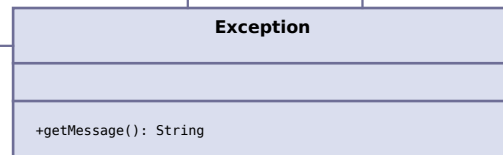
extends



throws

throws

throws



Hinweise zur Klasse Boardable (Einsteigefähig)

- Die Methode **getName** soll den Namen zurückgeben.

Hinweise zur Klasse SeatType (Sitztyp)

- Die Methode **getValue** soll den Wert (value) des Sitztyps zurückgeben.
- Die Methode **isPreferred** soll true zurückgeben, wenn der Sitztyp bevorzugt ist.
Ein Sitztyp ist bevorzugt, sofern es kein Mittel-Sitz (Middle) ist.

Hinweise zur Klasse Passenger (Passagier)

- **Konstruktor** soll alle Attribute initialisieren.
- Die Methode **getName** soll den Namen zurückgeben.

Hinweise zur Klasse Row (Reihe)

- **Konstruktor** soll alle Attribute initialisieren.
- Die Methode **getNextLetter** soll den Buchstaben des nächsten freien Platzes berechnen.
Der erste Platz hat den Buchstaben A, der zweite B, der dritte C und so weiter.
- Die Methode **isFull** soll ermitteln, ob die Reihe voll belegt ist.
Eine Reihe ist voll, sobald die Anzahl der Passagiere die maximale Anzahl (maxPassengers) erreicht hat.
- Die Methode **add** soll versuchen einen Passagier in der Reihe zu platzieren.
Ist die Reihe nicht voll, soll die Platzierung (Reihe und Buchstabe) auf der Konsole ausgegeben und der Passagier hinzugefügt werden. Andernfalls soll ein Fehler ausgelöst werden, dass die Reihe voll ist.

Hinweise zur Klasse FirstClassRow

- **Konstruktor** soll alle Attribute initialisieren.

Die First-Class-Reihe hat breitere Sitze und somit 2 Plätze weniger als die maximale Anzahl.

- Die Methode **add** soll versuchen, einen Passagier in der First-Class-Reihe zu platzieren.

Nur Flugbesatzung (FlightCrew) darf in der First-Class-Reihe platziert werden. Ist die zugewiesene Besatzung (assignedCrew) kleiner als die Mindestanzahl (MIN_CREW), darf niemand platziert werden.

Kann der Passagier nicht platziert werden, soll ein Fehler ausgelöst werden, dass nur Crew in der First Class erlaubt ist.

Hinweise zur Klasse EmergencyRow (Notausgangreihe)

- **Konstruktor** soll alle Attribute initialisieren.

Die Notausgangreihe soll immer in der untersten Reihe sein (Reihe 1).

- Die Methode **add** soll versuchen, einen Passagier in der Notausgangreihe zu platzieren.

Nur Passagiere (Passenger) deren Sitztyp bevorzugt ist, dürfen in der Notausgangreihe platziert werden.

Kann der Passagier nicht platziert werden, soll ein Fehler ausgelöst werden, dass nur Passagiere mit bevorzugtem Sitztyp erlaubt sind.

Hinweise zur Klasse Airplane (Flugzeug)

- **Konstruktor** soll alle Attribute initialisieren.

Jedes Flugzeug hat mehrere Reihen. Es gibt jedoch immer genau eine First-Class-Reihe und eine Notausgangreihe. Der Konstruktor soll nach der Initialisierung aller Attribute zuerst die First-Class-Reihe, dann die Notausgangreihe und anschliessend die normalen Reihen erzeugen und der internen Liste hinzufügen.

Die normalen Reihen werden in einem Loop erzeugt (Reihennummern absteigend). Die Notausgangreihe hat immer die geringste Reihe (Reihe 1) und die First-Class-Reihe immer die höchste Reihe. Die First-Class-Reihe hat 4 zugewiesene Crew-Mitglieder (assignedCrew).

Beispiel Liste mit numberOfRows = 3 und seatsPerRow = 3:

```
- FirstClassRow - Reihe = 5, max. 1 Passagier (3-2)
- EmergencyRow - Reihe = 1, max. 3 Passagiere
- normale Reihe - Reihe = 4, max. 3 Passagiere
- normale Reihe - Reihe = 3, max. 3 Passagiere
- normale Reihe - Reihe = 2, max. 3 Passagiere
```

Wenn ein Passagier platziert wird, erhält er automatisch den nächsten freien Buchstaben (A, B, C, ...).

- Die Methode **board** soll einen Passagier platzieren und zurückgeben, ob es funktioniert hat.

Versuche einen Platz fuer den Passagier im Flugzeug zu finden, indem ueber alle Reihen iteriert wird. Wenn ein Platz gefunden wurde, soll true zurueckgegeben werden.

Wenn der Passagier in einer Reihe nicht platziert werden konnte, soll die Fehlermeldung auf der Konsole ausgegeben und die naechste Reihe versucht werden.

Konnte der Passagier in keiner Reihe platziert werden, soll false zurueckgegeben werden.

Hinweise zur Klasse ExamTask

Es soll ein Flugzeug mit 4 Reihen und 3 Sitzen pro Reihe erstellt werden.

Erstelle ein Flugbesatzungsmitglied mit einem Namen deiner Wahl und versuche es im Flugzeug zu platzieren.

Iteriere ueber alle Sitztypen. Erstelle fuer jeden Sitztyp einen Passagier mit einem Alter von 25 und versuche ihn im Flugzeug zu platzieren. Verwende den Wert (getValue) des Sitztyps im Namen des Passagiers.

Falls kein Platz gefunden wurde, soll dies zusammen mit dem Wert des Sitztyps auf der Konsole ausgegeben werden.

Java API

Klasse	Methode	Statisch	Rueckgabotyp
SeatType	values()	X	SeatType[]
SeatType	name()		String
Exception	getMessage()		String
Boolean	valueOf(s: String)	X	Boolean
Boolean	valueOf(b: boolean)	X	Boolean
Double	valueOf(s: String)	X	Double
Double	valueOf(d: double)	X	Double
Integer	valueOf(s: String)	X	Integer
Integer	valueOf(i: int)	X	Integer
String	charAt(index: int)		char
String	length()		int

Java Collections Framework

Klasse	Methode	Statisch	Rueckgabotyp
ArrayList<T>	add(element: T)		boolean
ArrayList<T>	add(index: int, element: T)		void
ArrayList<T>	contains(element: T)		boolean
ArrayList<T>	get(index: int)		T
ArrayList<T>	remove(index: int)		T
ArrayList<T>	remove(element: T)		boolean
ArrayList<T>	size()		int
Collections	sort(list: List<T>)	X	void
Collections	sort(list, c: Comparator<T>)	X	void

Schnittstellen

Klasse	Methode	Statisch	Rueckgabotyp
Comparable<T>	compareTo(o: T)		int
Comparator<T>	compare(o1: T, o2: T)		int